Tracing

Capture detailed execution traces for debugging and analysis. Traces include DOM snapshots, screenshots, network activity, and console logs.

Basic Usage

```
# Start trace recording
playwright-cli tracing-start

# Perform actions
playwright-cli open https://example.com
playwright-cli click e1
playwright-cli fill e2 "test"

# Stop trace recording
playwright-cli tracing-stop
```

Trace Output Files

When you start tracing, Playwright creates a `traces/` directory with several files:

`trace-{timestamp}.trace`

Action log - The main trace file containing: - Every action performed (clicks, fills, navigations) - DOM snapshots before and after each action - Screenshots at each step - Timing information - Console messages - Source locations

`trace-{timestamp}.network`

Network log - Complete network activity: - All HTTP requests and responses - Request headers and bodies - Response headers and bodies - Timing (DNS, connect, TLS, TTFB, download) - Resource sizes - Failed requests and errors

`resources/`

Resources directory - Cached resources: - Images, fonts, stylesheets, scripts - Response bodies for replay - Assets needed to reconstruct page state

What Traces Capture

Category	Details
Actions	Clicks, fills, hovers, keyboard input, navigations
DOM	Full DOM snapshot before/after each action
Screenshots	Visual state at each step
Network	All requests, responses, headers, bodies, timing
Console	All console.log, warn, error messages
Timing	Precise timing for each operation

Use Cases

Debugging Failed Actions

```
playwright-cli tracing-start
playwright-cli open https://app.example.com

# This click fails - why?
playwright-cli click e5

playwright-cli tracing-stop
# Open trace to see DOM state when click was attempted
```

Analyzing Performance

```
playwright-cli tracing-start
playwright-cli open https://slow-site.com
playwright-cli tracing-stop

# View network waterfall to identify slow resources
```

Capturing Evidence

```
# Record a complete user flow for documentation
playwright-cli tracing-start

playwright-cli open https://app.example.com/checkout
playwright-cli fill e1 "4111111111111111"
playwright-cli fill e2 "12/25"
playwright-cli fill e3 "123"
playwright-cli click e4

playwright-cli tracing-stop
# Trace shows exact sequence of events
```

Trace vs Video vs Screenshot

Feature	Trace	Video	Screenshot
Format	.trace file	.webm video	.png/.jpeg image
DOM inspection	Yes	No	No
Network details	Yes	No	No
Step-by-step replay	Yes	Continuous	Single frame
File size	Medium	Large	Small
Best for	Debugging	Demos	Quick capture

Best Practices

1. Start Tracing Before the Problem

```
# Trace the entire flow, not just the failing step
playwright-cli tracing-start
playwright-cli open https://example.com
# ... all steps leading to the issue ...
playwright-cli tracing-stop
```

2. Clean Up Old Traces

Traces can consume significant disk space:

```
# Remove traces older than 7 days
find .playwright-cli/traces -mtime +7 -delete
```

Limitations

- Traces add overhead to automation
- Large traces can consume significant disk space
- Some dynamic content may not replay perfectly